

 Refactorisation réalisée

1. Création du CalendarManager

- **Nouveau fichier** : `js/managers/calendar-manager.js`
- **Classe créée** : `CalendarManager` qui encapsule toute la logique calendaire
- **Données migrées** : Variables globales du calendrier déplacées comme propriétés de classe

2. Fonctions migrées vers CalendarManager

- `loadCalendarFromCSV()` → `CalendarManager.loadCalendarFromCSV()`
- `saveCalendarToLocal()` → `CalendarManager.saveCalendarToLocal()`
- `loadCalendarFromLocal()` → `CalendarManager.loadCalendarFromLocal()`
- `updateCalendarUI()` → `CalendarManager.updateCalendarUI()`
- `updateDaySelector()` → `CalendarManager.updateDaySelector()`
- `updateCalendarDate()` → `CalendarManager.updateCalendarDate()`
- `exportCalendarToCSV()` → `CalendarManager.exportCalendarToCSV()`
- `updateSeasonDisplay()` → `CalendarManager.updateSeasonDisplay()`
- `setupSeasonListeners()` → `CalendarManager.setupSeasonListeners()`
- `loadSavedSeason()` → `CalendarManager.loadSavedSeason()`

3. Variables globales migrées

```
// De variables globales dans main.js

let currentSeason = 'printemps-debut';

let calendarData = null;

let currentCalendarDate = null;

let isCalendarMode = false;

const seasonSymbols = {...};

const seasonNames = {...};

// Vers propriétés de classe dans CalendarManager

this.currentSeason = 'printemps-debut';

this.calendarData = null;

this.currentCalendarDate = null;

this.isCalendarMode = false;
```

```
this.seasonSymbols = {...};
```

```
this.seasonNames = {...};
```

4. Interface publique créée

Méthodes getter/setter pour la compatibilité :

- `getCurrentSeason()`, `setCurrentSeason()`
- `getCalendarData()`, `setCalendarData()`
- `getCurrentCalendarDate()`, `setCurrentCalendarDate()`
- `getIsCalendarMode()`, `setIsCalendarMode()`

5. Intégration dans main.js

```
// Initialisation du manager

let calendarManager;

if (typeof CalendarManager !== 'undefined') {

    calendarManager = new CalendarManager();

    calendarManager.init();

}

// Remplacement des appels directs

// updateSeasonDisplay() → calendarManager.updateSeasonDisplay()

// loadSavedSeason() → calendarManager.loadSavedSeason()
```

6. Gestion des erreurs

Ajout de vérifications `typeof calendarManager !== 'undefined'` pour éviter les erreurs si le manager n'est pas chargé.

🎯 Résultat

- **Réduction** : ~400 lignes supprimées de `main.js`
- **Encapsulation** : Toute la logique calendaire isolée
- **Maintenabilité** : Code plus modulaire et organisé
- **Compatibilité** : Interface publique preserve le fonctionnement existant
- **Extensibilité** : Facilite l'ajout de nouvelles fonctionnalités calendaires